

Machine Learning (ML)

Block 4: Evaluation



Modellbewertung im Machine Learning

- **Frage:** Wie gut ist ein Modell für ein bestimmtes Problem?
 - Leistung des Modells bewerten
 - Ergebnisse überprüfen
 - Mit verschiedenen Modellen experimentieren
 - Bestes Modell auswählen
-

Einführung

- Verschiedene Klassifikationsalgorithmen bekannt (Decision Trees, Naive Bayes)
 - Es gibt viele weitere Algorithmen und Varianten (z. B. ID3, CART)
 - Algorithmen besitzen **Hyperparameter** (Einstellungen vor dem Training)
 - Beispiel: Pruning bei Decision Trees (min. Anzahl Daten pro Blatt)
 - Für ein Problem entstehen viele mögliche Modelle
 - **Frage:** Wie vergleicht man diese Modelle und findet das beste?
-

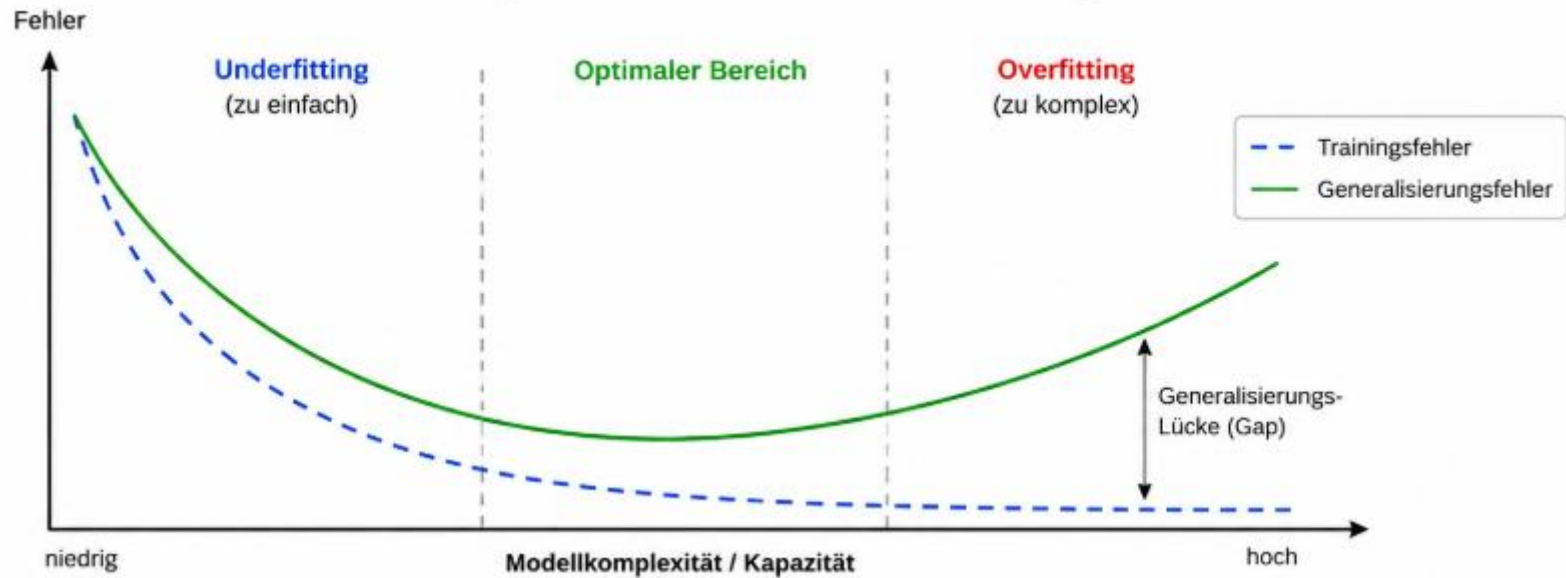
Einführung

- Performance = wie gut Vorhersagen zur echten Klasse passen
 - Fokus auf **Qualität**, nicht auf Laufzeit oder Speicher
 - Wichtigste Metrik: **Accuracy (Genauigkeit)** Accuracy = Anteil korrekt klassifizierter Daten
 - Error Rate = Anteil falsch klassifizierter Daten
 - Zusammenhang: **Accuracy = 1 – Error Rate**
-

Trainingsfehler vs. Generalisierungsfehler

- Accuracy und Error Rate hängen vom Datensatz ab
 - **Training Error:** Fehler auf Trainingsdaten
 - **Generalization Error:** Fehler auf neuen/unbekannten Daten
 - Wichtig: **Generalization Error** ist entscheidend
 - Schätzung z. B. mit **Holdout-Methode**
-

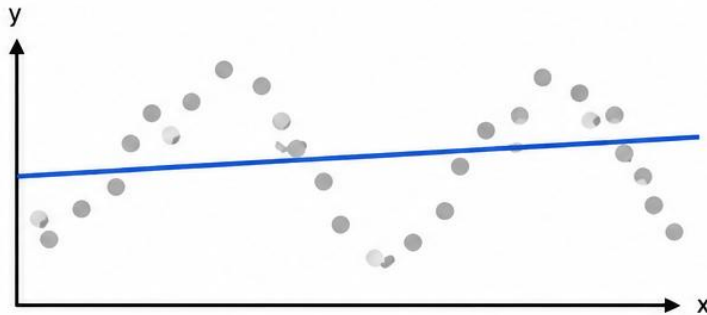
Trainingsfehler und Generalisierungsfehler



Trainingsfehler vs. Generalisierungsfehler

Ziel: Ein Modell soll Muster in den Trainingsdaten lernen und auf neue Daten gut übertragbar sein.

Underfitting (hoher Bias)

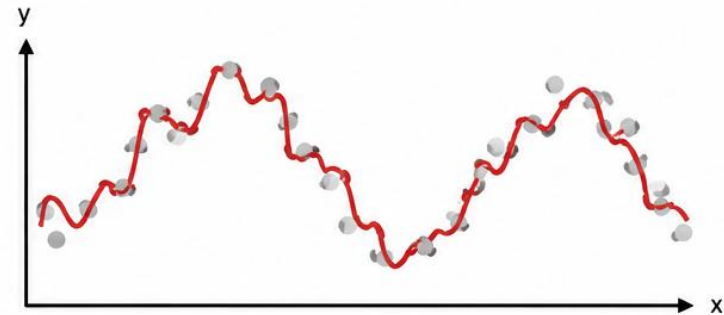


● Datenpunkte — Modell (zu einfach)

- Hoher Trainingsfehler
- Hoher Generalisierungsfehler

Das Modell ist **zu einfach**.
Es erfasst das Muster nicht gut.

Overfitting (hohe Varianz)



● Datenpunkte — Modell (zu komplex)

- Niedriger Trainingsfehler
- Hoher Generalisierungsfehler

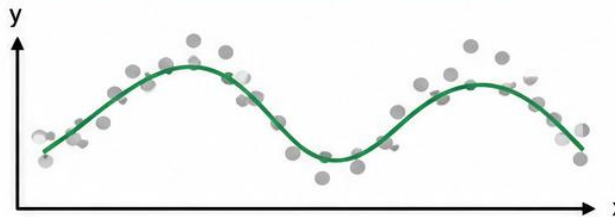
Das Modell ist **zu komplex**.
Es passt sich dem Rauschen an.



Gut generalisiert

- Niedriger Trainingsfehler
- Niedriger Generalisierungsfehler

Das Modell ist passend komplex.
Es erfasst das Muster und generalisiert gut.



Trainingsfehler vs. Generalisierungsfehler

Holdout-Methode

- Datensatz wird **einmal** aufgeteilt (z. B. 80% Training, 20% Test)
 - Modell wird auf Trainingsdaten trainiert
 - Bewertung auf Testdaten
 - Einfach und schnell
 - Nachteil: Ergebnis hängt stark von der Aufteilung ab
-

Extremes Beispiel: Ergebnis hängt extrem von der Aufteilung ab



Ziel: Vorhersage, ob ein Kunde ein Produkt kauft (Ja/Nein).
Wir haben 10 Kunden und 2 Attribute.

- Attribut 1: Alter (jung < 30 / alt ≥ 30)
- Attribut 2: Einkommen (niedrig < 3000€ / hoch ≥ 3000€)

Gesamter Datensatz (10 Kunden)

ID	Alter	Einkommen	Kauf (Ja/Nein)
1	jung	hoch	Ja
2	jung	hoch	Ja
3	jung	hoch	Ja
4	jung	hoch	Ja
5	jung	hoch	Ja
6	alt	niedrig	Nein
7	alt	niedrig	Nein
8	alt	niedrig	Nein
9	alt	niedrig	Nein
10	alt	niedrig	Nein



Sehr einfaches Modell (Regel):

Wenn mindestens 3 von den Trainingsbeispielen „Kauf = Ja“ sind,
sagt das Modell für neue Kunden → „Kauf = Ja“,
sonst → „Kauf = Nein“.

Im gesamten Datensatz: 5 mal „Ja“, 5 mal „Nein“

Aufteilung A – Extrem gutes Ergebnis

70% Training (7 Kunden) / 30% Test (3 Kunden)

Training (7 Kunden)

ID	Alter	Einkommen	Kauf
1	jung	hoch	Ja
2	jung	hoch	Ja
3	jung	hoch	Ja
4	jung	hoch	Ja
5	jung	hoch	Ja
6	alt	niedrig	Nein
7	alt	niedrig	Nein

Test (3 Kunden)

ID	Alter	Einkommen	Kauf (tatsächlich)
8	alt	niedrig	Nein
9	alt	niedrig	Nein
10	alt	niedrig	Nein

Im Training:

- Ja: 5, Nein: 2 → $5 \geq 3$
- Modell sagt: „Kauf = Ja“

Vorhersage auf Testdaten:

- Für ID 8: vorhergesagt Ja, tatsächlich Nein ✗
- Für ID 9: vorhergesagt Ja, tatsächlich Nein ✗
- Für ID 10: vorhergesagt Ja, tatsächlich Nein ✗

Ergebnis Aufteilung A: 0 von 3 richtig (0% Genauigkeit) ✗

Aufteilung B – Extrem schlechtes Ergebnis

70% Training (7 Kunden) / 30% Test (3 Kunden)

Training (7 Kunden)

ID	Alter	Einkommen	Kauf
6	alt	niedrig	Nein
7	alt	niedrig	Nein
8	alt	niedrig	Nein
9	alt	niedrig	Nein
10	alt	niedrig	Nein
1	jung	hoch	Ja
2	jung	hoch	Ja

Test (3 Kunden)

ID	Alter	Einkommen	Kauf (tatsächlich)
3	jung	hoch	Ja
4	jung	hoch	Ja
5	jung	hoch	Ja

Im Training:

- Ja: 2, Nein: 5 → $2 < 3$
- Modell sagt: „Kauf = Nein“

Vorhersage auf Testdaten:

- Für ID 3: vorhergesagt Nein, tatsächlich Ja ✗
- Für ID 4: vorhergesagt Nein, tatsächlich Ja ✗
- Für ID 5: vorhergesagt Nein, tatsächlich Ja ✗

Ergebnis Aufteilung B: 0 von 3 richtig (0% Genauigkeit) ✗



Fazit: Gleicher Datensatz, gleiches Modell – aber je nachdem, welche Kunden im Test landen,
erhält man ein extrem unterschiedliches Ergebnis (hier: von 0% bis 0%, in anderen Fällen z.B. 100% vs. 0%).
→ Das Ergebnis hängt extrem stark von der Aufteilung ab!

Trainingsfehler vs. Generalisierungsfehler

Random Subsampling

- Mehrfaches zufälliges Aufteilen in Trainings- und Testdaten
 - Modell wird **mehrmals trainiert und getestet**
 - Ergebnisse werden gemittelt
 - Stabiler als Holdout
 - Nachteil: Daten können mehrfach oder gar nicht im Testset sein
-

Trainingsfehler vs. Generalisierungsfehler

Random Subsampling

- Mehrfaches zufälliges Aufteilen in Trainings- und Testdaten
 - Modell wird **mehrmals trainiert und getestet**
 - Ergebnisse werden gemittelt
 - Stabiler als Holdout
 - Nachteil: Daten können mehrfach oder gar nicht im Testset sein
-

Trainingsfehler vs. Generalisierungsfehler – Random Subsampling

Idee: Wir teilen die Daten mehrmals zufällig in Trainings- und Testdaten auf, trainieren und testen das Modell und mitteln die Ergebnisse.

Datensatz (10 Kunden, 2 Attribute)

ID	Alter	Einkommen	Kauf (Ja/Nein)
1	jung	hoch	Ja
2	jung	hoch	Ja
3	jung	niedrig	Nein
4	alt	hoch	Ja
5	alt	hoch	Ja
6	alt	niedrig	Nein
8	alt	niedrig	Nein
9	alt	hoch	Ja
10	alt	niedrig	Nein

Einfaches Modell (Regel):

Wenn mindestens 3 von den Trainingsbeispielen „Kauf = Ja“ sind, sagt das Modell für neue Kunden „Kauf = Ja“, sonst „Kauf = Nein“.

Im gesamten Datensatz: 5 mal „Ja“, 5 mal „Nein“

Aufteilung 1

70% Training (7) / 30% Test (3)

Training (7 Kunden)

1	Ja
2	Ja
3	Nein
4	Ja
5	Ja
6	Nein
7	Nein

Test (3 Kunden)

	tatsächlich	Vorhersage
8	Nein	Nein ✓
9	Ja	Ja ✓
10	Nein	Nein ✓

Ergebnis Aufteilung 1: 3 von 3 richtig (100%)

Aufteilung 2

70% Training (7) / 30% Test (3)

Training (7 Kunden)

1	Ja
2	Ja
3	Nein
4	Ja
5	Ja
6	Nein
8	Nein

Test (3 Kunden)

	tatsächlich	Vorhersage
7	Nein	Nein ✓
9	Ja	Nein ✗
10	Nein	Nein ✓

Ergebnis Aufteilung 2: 2 von 3 richtig (66,7%)

Aufteilung 3

70% Training (7) / 30% Test (3)

Training (7 Kunden)

1	Ja
2	Ja
4	Ja
5	Ja
6	Nein
7	Nein
8	Nein

Test (3 Kunden)

	tatsächlich	Vorhersage
3	Nein	Nein ✓
9	Ja	Nein ✗
10	Nein	Nein ✓

Ergebnis Aufteilung 3: 2 von 3 richtig (66,7%)



Ergebnisse werden gemittelt:

$(100\% + 66,7\% + 66,7\%) / 3 = 77,8\%$

→ **Geschätzter Generalisierungsfehler: 22,2%**



Warum besser als Holdout?

- Weniger abhängig von einer einzigen Aufteilung
- Stabilere und zuverlässigere Schätzung



Nachteil

- Daten können mehrfach im Testset oder gar nicht im Testset sein.



Fazit: Random Subsampling ist stabiler als ein einzelnes Holdout-Verfahren, nutzt die Daten aber nicht so effizient wie z. B. k-Fold Cross-Validation.

Trainingsfehler vs. Generalisierungsfehler

Cross-Validation



Der Datensatz wird in $k = 5$ gleich große **Teilmenzen** aufgeteilt.

- In jedem Durchlauf:
1 Teilmenge = **Testdaten**
 $k - 1$ Teilmenzen = Trainingsdaten
- Der Vorgang wird k -mal wiederholt.
- Jede Teilmenge dient genau **einmal** als Testset.
- Ergebnis = **Durchschnitt** aller Durchläufe.

Teilmenzen des Datensatzes

	Teilmenge 1	Teilmenge 2	Teilmenge 3	Teilmenge 4	Teilmenge 5
Durchlauf 1	Testdaten	Trainingsdaten	Trainingsdaten	Trainingsdaten	Trainingsdaten
Durchlauf 2	Trainingsdaten	Testdaten	Trainingsdaten	Trainingsdaten	Trainingsdaten
Durchlauf 3	Trainingsdaten	Trainingsdaten	Testdaten	Trainingsdaten	Trainingsdaten
Durchlauf 4	Trainingsdaten	Trainingsdaten	Trainingsdaten	Testdaten	Trainingsdaten
Durchlauf 5	Trainingsdaten	Trainingsdaten	Trainingsdaten	Trainingsdaten	Testdaten



Testdaten (1 Teilmenge)



Trainingsdaten ($k - 1$ Teilmenzen)



Ergebnis:

Durchschnitt aller Durchläufe

Class Imbalance Problem

- **Class Imbalance Problem:** Klassen sind ungleich verteilt
 - Beispiel: 98% „nicht Spam“, 2% „Spam“
 - Modell lernt vor allem die **häufige Klasse**
 - Seltene Klasse wird oft **schlecht erkannt**
 - Accuracy kann **irreführend hoch** sein
 - Beispiel: Modell sagt immer „nicht Spam“ → Accuracy = 98%
 - Problem: Wichtige Fälle (z. B. Spam) werden übersehen
-

Class Imbalance Problem – Beispiel

- Wenn ein Modell immer „nicht Spam“ vorhersagt und 98% der Daten „nicht Spam“ sind – wie aussagekräftig ist dann die Genauigkeit?

1. Verteilung im Datensatz



Nicht Spam

980 E-Mails (98%)



Spam

20 E-Mails (2%)

Gesamt: 1.000 E-Mails

2. Modellverhalten

Das Modell sagt für jede E-Mail:

„Nicht Spam“



⋮



Vorhersage:
immer
„Nicht Spam“

Class Imbalance Problem

Confusion Matrix

- True Positive (TP): Modell sagt positiv und es ist tatsächlich positiv.
- False Negative (FN): Modell sagt negativ, obwohl es eigentlich positiv ist (übersehen).
- False Positive (FP): Modell sagt positiv, obwohl es negativ ist (Fehlalarm).
- True Negative (TN): Modell sagt negativ und es ist tatsächlich negativ.

		Vorhergesagte Klasse	
		Positive Klasse	Negative Klasse
Tatsächliche Klasse	Positive Klasse	 True Positive (TP)	 False Negative (FN)
	Negative Klasse	 False Positive (FP)	 True Negative (TN)

Confusion Matrix

Aufgabe: Ein Modell sagt für jede E-Mail „Nicht Spam“ voraus.

Wie viele True Positives (TP), True Negatives (TN), False Negatives (FN) und False Positives (FP) haben wir?

1. Verteilung im Datensatz



Nicht Spam

980 E-Mails (98%)



Spam

20 E-Mails (2%)

Gesamt: 1.000 E-Mails

2. Modellverhalten

Das Modell sagt für jede E-Mail:

„Nicht Spam“



⋮



Vorhersage:
immer
„Nicht Spam“

Wie viele TP, TN, FN und FP haben wir?

TP =

TN =

FN =

FP =

TP = True Positive
(Spam richtig als Spam erkannt)

TN = True Negative
(Nicht Spam richtig als Nicht Spam erkannt)

FN = False Negative
(Spam fälschlich als Nicht Spam erkannt)

FP = False Positive
(Nicht Spam fälschlich als Spam erkannt)

Confusion Matrix

Bewertungsmetriken



1. Recall (Trefferquote / True Positive Rate)

- Anteil der tatsächlich positiven Fälle, die korrekt erkannt werden.
 - **Fokus:** Wie viel wird gefunden?
- Wichtig, wenn Übersehen kritisch ist (z. B. Spam, Betrug).

Formel

$$\text{Recall} = \frac{TP}{TP + FN}$$



2. Precision (Genauigkeit)

- Anteil der als positiv klassifizierten Fälle, die wirklich positiv sind.
 - **Fokus:** Wie verlässlich ist „positiv“?
- Wichtig, wenn Fehlalarme teuer sind.

Formel

$$\text{Precision} = \frac{TP}{TP + FP}$$



3. False Positive Rate (FPR)

- Anteil negativer Fälle, die fälschlich als positiv eingestuft werden.
 - **Fokus:** Wie oft gibt es unnötige Alarme?
- Wichtig zur Kontrolle von Fehlalarmen.

Formel

$$\text{FPR} = \frac{FP}{FP + TN}$$

Confusion Matrix

Aufgabe:

Ein Unternehmen entwickelt ein System zur Erkennung von Spam-E-Mails.

In einem Testdatensatz befinden sich 10.000 E-Mails:

- 9.800 sind kein Spam (98 %)
- 200 sind Spam (2 %)

Das Modell wurde auf diesen Datensatz angewendet.

Das Modell klassifiziert jede E-Mail als „Kein Spam“.

Bestimmen Sie anhand der Beschreibung die Werte für:

- True Positive (TP) = _____
- False Negative (FN) = _____
- False Positive (FP) = _____
- True Negative (TN) = _____

Bestimmen Sie Recall, Precision, FPR, Accuracy und Error Rate.

Konfusionsmatrix

	Vorhergesagt: Spam	Vorhergesagt: Kein Spam
Tatsächlich: Spam	True Positive (TP)	False Negative (FN)
Tatsächlich: Kein Spam	False Positive (FP)	True Negative (TN)

Gegeben:

- Tatsächlich Spam = 200
- Tatsächlich Kein Spam = 9.800
- Modell klassifiziert jede E-Mail als „Kein Spam“

Threshold (Schwellenwert)

- Viele Modelle geben keine festen Klassen aus, sondern eine Wahrscheinlichkeit (z. B. „Spam = 0.7“).
 - Der Threshold bestimmt dann:
 - $\geq$ Threshold $\rightarrow$ positiv (z. B. Spam)
 - $<$ Threshold $\rightarrow$ negativ (kein Spam)
 - Beispiel:
 - Threshold = **0.5**
 - Modell sagt: Spam-Wahrscheinlichkeit = **0.7** $\rightarrow$ **Spam**
 - Modell sagt: Spam-Wahrscheinlichkeit = **0.3** $\rightarrow$ **kein Spam**
-

Threshold (Schwellenwert)

Aufgabe

Aufgabe: Einfluss des Thresholds

- Denken Sie darüber nach, wie die Wahl des Thresholds die Klassifikation beeinflusst.

Fragestellung:

- In welchen Situationen sollte der Threshold **hoch** gesetzt werden?
 - In welchen Situationen sollte der Threshold **niedrig** gesetzt werden?
-

Cost-sensitive Evaluation

- In vielen Anwendungen kann man **Kosten für jede Entscheidung** in der Konfusionsmatrix festlegen.
- Beim Spam-Filter könnte man z. B. folgende Kosten annehmen:

	Vorhergesagt: Spam	Vorhergesagt: Kein Spam
Tatsächlich Spam	Spam erkannt → 0 € (TP)	Spam übersehen → 80 € (FN)
Tatsächlich kein Spam	Normale Mail blockiert → 30 € (FP)	Korrekt zugestellt → 0 € (TN)

Cost-sensitive Evaluation

Aufgabe: Betrugserkennung (Fraud Detection)

Ein Unternehmen entwickelt ein Modell zur Erkennung von betrügerischen Transaktionen.

In einem Testdatensatz befinden sich **10.000 Transaktionen**:

- **9.700** sind normal (kein Betrug)
- **300** sind Betrug

Modell A

Threshold = 0.5			
		Vorhergesagte Klasse	
		Betrug (positiv)	Kein Betrug (negativ)
Tatsächliche Klasse	Betrug (positiv)	TP = 210	FN = 90
	Kein Betrug (negativ)	FP = 300	TN = 9.400
Summe: $210 + 90 + 300 + 9.400 = 10.000$			

Threshold = 0.2			
		Vorhergesagte Klasse	
		Betrug (positiv)	Kein Betrug (negativ)
Tatsächliche Klasse	Betrug (positiv)	TP = 270	FN = 30
	Kein Betrug (negativ)	FP = 1.000	TN = 8.700
Summe: $270 + 30 + 1.000 + 8.700 = 10.000$			

Modell B

Threshold = 0.5			
		Vorhergesagte Klasse	
		Betrug (positiv)	Kein Betrug (negativ)
Tatsächliche Klasse	Betrug (positiv)	TP = 240	FN = 60
	Kein Betrug (negativ)	FP = 600	TN = 9.100
Summe: $240 + 60 + 600 + 9.100 = 10.000$			

Threshold = 0.2			
		Vorhergesagte Klasse	
		Betrug (positiv)	Kein Betrug (negativ)
Tatsächliche Klasse	Betrug (positiv)	TP = 285	FN = 15
	Kein Betrug (negativ)	FP = 1.500	TN = 8.200
Summe: $285 + 15 + 1.500 + 8.200 = 10.000$			

Kosten für Fehler

- False Negative (FN): **200 €**
- False Positive (FP): **10 €**
- True Positive (TP): **0 €**
- True Negative (TN): **0 €**

Aufgaben

- 1 Berechnen Sie für jede Kombination (Modell + Threshold) die Gesamtkosten.
- 2 Welcher Threshold ist für jedes Modell besser?
- 3 Welches Modell ist insgesamt besser?
- 4 Begründen Sie Ihre Entscheidung.

Quellen

- Volker Ahlers. Machine Learning. Lehrveranstaltungsskript. Interne Quelle. 2023/2024. url: <https://moodle.hs-hannover.de/course/view.php?id=24225>.
- https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html

